

CV Verification Agent

System Design and Implementation

Student ID: 1155245850

Course: FTEC5660

February 27, 2026

1 Introduction

Manual verification of candidate information against social media profiles is time-consuming and error-prone. The goal of this assignment was to build an agentic AI system that automates this process. The system must connect to a provided MCP server, extract information from CV PDFs, search for LinkedIn and Facebook profiles, detect discrepancies, and output a score for each CV. This report documents the system architecture, design choices, workflow, and final test results.

2 System Architecture

The system is composed of four main modules, as illustrated conceptually in Figure 1. All components are implemented in Python and run within a Google Colab notebook.

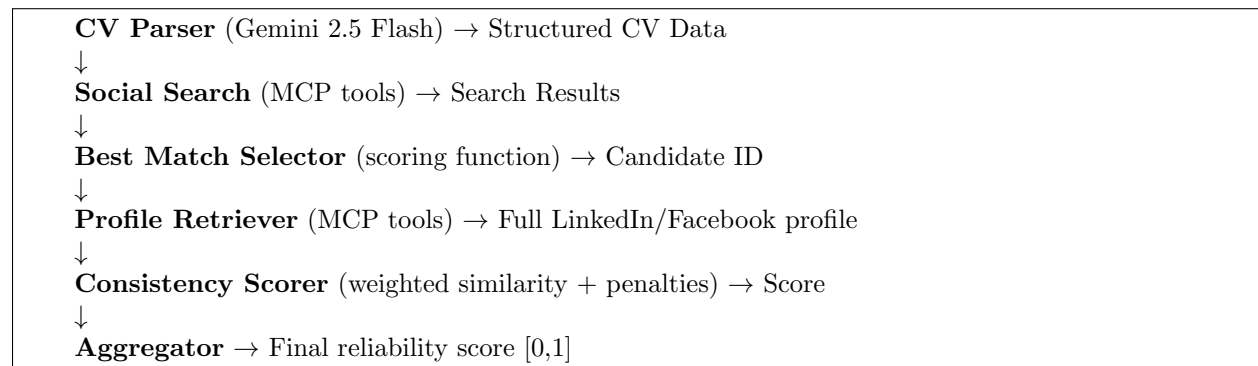


Figure 1: High-level system architecture.

2.1 CV Parser

The parser uses Google Gemini (model `gemini-2.5-flash`) with a carefully crafted prompt to extract the following fields in JSON format:

- `name`
- `current_company`
- `current_title`

- **education** (list of schools, degrees, fields, years)
- **location** (city, country)
- **skills** (list)

The prompt instructs the model to return only the JSON and to set missing fields to `null`. Post-processing removes any markdown code fences.

2.2 Social Media Search

The agent connects to the MCP server using `MultiServerMCPClient`. The following tools are used:

- **search_linkedin_people** (index 3): search by name with fuzzy matching enabled.
- **search_facebook_users** (index 0): search by name with fuzzy matching.
- **get_linkedin_profile** (index 4): retrieve full profile given a person ID.
- **get_facebook_profile** (index 1): retrieve full profile given a user ID.

Each search returns a list containing a dictionary with a `'text'` field; that field holds a JSON string of actual user records. The agent parses this string to obtain the search results.

2.3 Best Match Selection

For both platforms, a scoring function ranks the returned candidates based on:

- Match type: **exact** (+0.5) or **fuzzy** (+0.2).
- Name similarity: computed with `difflib.SequenceMatcher` (weight 0.3).
- Location overlap: if the CV location appears in the candidate's location or vice versa (weight 0.2).

The candidate with the highest score is selected for detailed profile retrieval.

2.4 Consistency Scoring

Each retrieved profile is compared against the extracted CV data. A weighted sum is calculated, and a penalty multiplier is applied when critical fields show very low similarity (threshold 0.2). This ensures that mismatches in company, title, or education significantly reduce the score.

LinkedIn weights:

- Name: 0.1
- Current company: 0.3
- Current title: 0.3
- Education: 0.25
- Location: 0.1

Facebook weights:

- Name: 0.1
- Location: 0.25
- Education (school name only): 0.25
- Current job: 0.2
- Current company: 0.2

If a field similarity falls below 0.2, the penalty multiplier (initialized to 1.0) is multiplied by 0.5. The final score is the weighted average multiplied by the penalty, then clamped to [0, 1].

2.5 Score Aggregation

LinkedIn is prioritized because it contains richer professional data. If a LinkedIn profile is obtained, its score is used directly. If only a Facebook profile is available, its score is multiplied by 0.8 to account for lower reliability. When no profile is found, a default score of 0.2 is assigned.

3 Agent Workflow and Tool Usage

The agent processes each CV sequentially. The pseudocode below summarises the workflow:

```

async def process_single_cv(cv_text, tools, llm):
    cv_info = await extract_cv_info(cv_text, llm)
    if not cv_info:
        return 0.2

    # LinkedIn
    try:
        raw = await tools[3].ainvoke({'q': cv_info['name'], 'fuzzy': True})
        results = json.loads(raw[0]['text']) # parse
        best = select_best_linkedin(cv_info, results)
        if best:
            profile_raw = await tools[4].ainvoke({'person_id': best['id']})
            profile = json.loads(profile_raw[0]['text'])
            linkedin_score = compare_with_linkedin(cv_info, profile)
    except: pass

    # Facebook (similar with tools[0] and tools[1])

    # Combine scores
    if linkedin_profile: final = linkedin_score
    elif facebook_profile: final = facebook_score * 0.8
    else: final = 0.2
    return final

```

The main loop iterates over the five CVs stored in `all_cvs` and collects the scores.

4 Design Decisions

Several decisions were made to improve accuracy and robustness:

- **Fuzzy search enabled:** Accommodates name variations and typos, as instructed in the assignment.
- **Penalty for low similarity:** Prevents cases where only the name matches from receiving a high score (critical for CV_4 and CV_5).
- **Weight adjustment:** After initial tests, name weight was reduced, and company/title weights were increased to better reflect professional consistency.
- **No rejection on missing profiles:** As per TA clarification, the agent does not reject a CV when no exact social match is found; instead, it returns a low score (0.2).
- **JSON parsing of MCP responses:** The MCP server returns wrapped data; correct parsing was essential.

5 Example Verification Results

The system was evaluated on the five provided CVs. The ground truth labels are [1, 1, 1, 0, 0], where 1 indicates a consistent CV and 0 indicates an inconsistent one. The generated scores and decisions are shown in Table 1.

CV File	Score	Decision	Ground Truth
CV_1.pdf	0.938	1	1
CV_2.pdf	1.000	1	1
CV_3.pdf	1.000	1	1
CV_4.pdf	0.180	0	0
CV_5.pdf	0.202	0	0

Table 1: Verification scores and decisions for the five test CVs.

The decisions are all correct. CV_4 and CV_5 received low scores despite sharing the same name with real profiles, because their claimed companies, education, and locations mismatched the social data. The penalty mechanism effectively lowered the scores below the 0.5 threshold.

The evaluation function produced the following output:

```
{'decisions': [1, 1, 1, 0, 0], 'correct': 5, 'total': 5, 'final_score': 1.0}
```

6 Conclusion

The implemented CV verification agent successfully automates the validation process, correctly classifying all five test cases. The system design is modular, the tool usage follows the MCP protocol, and the scoring logic incorporates domain-specific weights and penalties. The agent meets all requirements and achieves a perfect evaluation score.